AgentSQL: A Scalable and Resilient Asymmetric LLM Pipeline for Text-to-SQL Workflows

Hoang-Khang Nguyen, **Ngoc-Phuong-Dung Tran**, **Ha-Phuong Pham**, and **Quang-Khai Nguyen**
Master of Data Science Program, University of Science, VNU-HCM, Vietnam
{25c0103580, 25c0102837, 25c0104350, 25c0100785}@student.hcmus.edu.vn

Abstract—Translating natural language into executable SQL (Text-to-SQL) on enterprise-scale databases remains challenging due to schema ambiguity, error-type conflation, and prohibitive LLM inference costs. We present AgentSQL, an asymmetric multi-agent pipeline that addresses these challenges through three key innovations: (1) a four-phase architecture integrating CHESS-based FAISS semantic pruning, MCI-SQL metadata enrichment, and MAGIC self-correction guidelines with a strict cost invariant of at most three LLM API calls per question; (2) a Reflector–Critic asymmetric loop that decouples lightweight logical validation (back-translation consistency) from high-reasoning syntactical correction, invoking the expensive Critic model only when errors are detected; and (3) a three-tier Semantic State Evaluation layer that classifies execution outcomes into SyntaxError, EmptyResult, and NullResult states, each carrying actionable correction suggestions. A round-robin key-rotation mechanism with zero-sleep failover enables sustained throughput during batch evaluation. Preliminary experiments on the BIRD Mini-Dev benchmark demonstrate 80% Execution Accuracy with an average of 2.0 API calls per question, validating the cost-efficiency of the asymmetric model allocation strategy.

Source Code: https://github.com/khang3004/AgentSQL-Asym.git

Index Terms—Text-to-SQL, Large Language Models (LLMs), Multi-Agent Systems, Asymmetric Pipeline, Semantic Schema Pruning, Metadata Enrichment, Self-Correction Guidelines, BIRD Benchmark, Execution Accuracy, Resilient AI Infrastructure.

1 Introduction

Translating natural language into executable Structured Query Language (Text-to-SQL) has emerged as a pivotal capability for enabling non-expert users to interrogate large-scale relational databases. Despite remarkable progress driven by Large Language Models (LLMs), the gap between research prototypes and production-grade interfaces remains significant. Current systems face three interrelated challenges:

1) **Schema Ambiguity**: Enterprise databases routinely contain hundreds of tables with cryptic naming conventions, causing LLMs to hallucinate non-existent columns or misinterpret join paths (Floratou et al., 2024; Li et al., 2023).
2) **Error Taxonomy**: Existing correction mechanisms conflate syntactical errors (detectable by the database engine) with logical errors (semantically incorrect but syntactically valid queries). A unified correction prompt wastes reasoning capacity on trivially diagnosable failures.
3) **Cost–Accuracy Trade-off**: Monolithic frontier-model approaches (e.g., GPT-4, Claude 3) achieve high accuracy but incur prohibitive per-query costs, making large-scale batch evaluation economically infeasible.

To address these challenges, we present **AgentSQL**, an asymmetric multi-agent pipeline that decouples schema understanding from SQL generation and, crucially, separates logical validation from syntactical correction. AgentSQL integrates three complementary upstream modules—CHESS-based semantic pruning (Talaei et al., 2024), MCI-SQL metadata enrichment (Q. Wang et al., 2026), and MAGIC self-correction guidelines (Askari et al., 2024)—into a unified four-phase pipeline orchestrated by a `MasterPipeline` class. At its core lies an *Asymmetric LLM Loop*: a lightweight **Reflector** performs back-translation-based logical verification at near-zero latency, while a high-reasoning **Resilient Critic** is invoked *only* when errors are detected, keeping API costs bounded to at most three calls per question.

Contributions. Our main contributions are:

- A modular, four-phase pipeline architecture with strict cost invariants (≤ 3 LLM API calls per question) and full LangSmith observability.
- A three-tier Semantic State Evaluation layer (`SemanticErrorChecker`) that classifies execution outcomes into SyntaxError, EmptyResult, and NullResult states, each carrying an actionable correction suggestion.
- A round-robin `KeyRotator` abstraction supporting N API keys per provider with zero-sleep rotation on HTTP 429, enabling sustained high-throughput batch evaluation.
- Preliminary evaluation on the BIRD mini-dev benchmark demonstrating 80% Execution Accuracy with an average of 2.0 API calls per question.

2 Related Work

2.1 Text-to-SQL Benchmarks and Their Limitations

The transition from academic-scale datasets (Spider, WikiSQL) to industry-representative benchmarks has been catalysed by the **BIRD** benchmark (Li et al., 2023), which introduces 12,751 question–SQL pairs across 95 large-scale databases containing real-world dirty data. BIRD requires models to reason over complex multi-table joins, date format variations, and domain-specific value conventions. However, recent audits

have revealed that up to 10% of BIRD annotations contain semantic ambiguities or outright errors (Floratou et al., 2024), underscoring the need for systems that remain resilient to imperfect evaluation signals.

2.2 Schema Pruning and Context Construction

Providing an LLM with a full database schema—potentially hundreds of tables—exceeds context windows and degrades reasoning quality. **CHESS** (Talaei et al., 2024) proposed a two-phase embedding pipeline:

1) **Offline**: Index all table representations into a FAISS vector store (Johnson et al., 2021) using sentence-transformer embeddings (Xiao et al., 2024).
2) **Online**: Embed only the user question and retrieve the Top-k most similar tables via inner-product search, achieving sub-millisecond latency.

Building on pruned schemas, **MCI-SQL** (Q. Wang et al., 2026) demonstrated that enriching prompts with *multiple facets* of context information—column data types, cardinality ratios (1:1 PK vs. 1:N FK), MIN/MAX statistics, and sample values—significantly improves schema linking accuracy. AgentSQL integrates both strategies sequentially: CHESS prunes the schema from $|\mathcal{S}|$ to k tables, then MCI-SQL enriches the pruned subset with column-level metadata.

2.3 Multi-Agent and Self-Correction Paradigms

Multi-agent frameworks decompose the Text-to-SQL task into specialised sub-tasks. **MAC-SQL** (B. Wang et al., 2025) employs a Decomposer, a Selector, and a Refiner operating collaboratively. **MAGIC** (Askari et al., 2024) generates self-correction guidelines from in-context exemplars, providing the LLM with an explicit error-checklist during generation. Execution-feedback approaches—**ReViSQL** (Zhu et al., 2026) and **AGENTIQL** (Heidari et al., 2025)—iterate between generation and database execution to fix queries empirically.

AgentSQL distinguishes itself from these approaches along two axes:

- **Asymmetric model allocation**: Unlike MAC-SQL's homogeneous agent pool, AgentSQL assigns a cost-efficient model (`llama-4-scout-17b`) for generation and reflection, reserving a high-reasoning model (`gemini-2.5-flash`) exclusively for the correction phase.
- **Error-type isolation**: Rather than sending all failures to a single corrector, a lightweight Reflector catches *logical* mismatches (via back-translation) before execution, while a three-tier `SemanticErrorChecker` classifies *syntactical* and *semantic* failures post-execution—each with tailored correction suggestions.

3 Methodology

3.1 Problem Formulation

We formulate the Text-to-SQL task over a relational database $\mathcal{D} = (\mathcal{S}, \mathcal{V})$, where $\mathcal{S} = \{T_1, T_2, \ldots, T_M\}$ is the set of M tables comprising the schema and $\mathcal{V}$ denotes the stored data values. Each table T_j is characterised by its column set $\{c_{j,1}, \ldots, c_{j,|T_j|}\}$. Given a natural language question Q and optional evidence hint E, the objective is to generate the optimal SQL query:

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} P_\theta(y \mid Q, \mathcal{S}', \mathbf{C}_{\text{meta}}, E) \tag{1}$$

where $\mathcal{S}' \subseteq \mathcal{S}$ is the CHESS-pruned schema ($|\mathcal{S}'| = k \ll M$), $\mathbf{C}_{\text{meta}}$ is the MCI metadata context, and θ denotes the LLM parameters.

3.2 Architecture Overview

As illustrated in Figure 1, the `MasterPipeline` orchestrates four sequential phases with a strict cost invariant:

> **Design Invariant**: A maximum of **3 LLM API calls** per question—Generation (call #1), Reflection (call #2), and conditional Correction (call #3). All schema pruning, metadata extraction, and semantic validation are executed locally at zero API cost.

3.3 Phase 1: CHESS Semantic Pruning

Given the question Q, we embed it using the BGE asymmetric query prefix and retrieve the Top-k tables from a pre-built FAISS index (Johnson et al., 2021; Xiao et al., 2024):

$$\mathcal{S}' = \text{Top-k}_{T_j \in \mathcal{S}} \left\langle \mathbf{e}_Q,\ \mathbf{e}_{T_j} \right\rangle \tag{2}$$

where $\mathbf{e}_Q = f_\theta(\texttt{"Represent this sentence: "} \| Q)$ is the L2-normalised query embedding and $\mathbf{e}_{T_j}$ is the pre-computed table representation stored in the FAISS `IndexFlatIP`. The index is built *once offline* by `build_offline_index.py`, encoding each table as a concatenation of its name, column names, and data-type declarations. This reduces the schema from M tables to k (default $k = 3$), yielding a typical reduction ratio of 60–80%.

3.4 Phase 2: MCI-SQL Metadata Enrichment

The `MetadataExtractor` profiles each column $c \in \mathcal{S}'$ locally via SQLite PRAGMA introspection and targeted queries:

- **Numeric columns**: Extracts $\min(c)$ and $\max(c)$ for range validation.
- **Text columns**: Fetches $n_s = 3$ random non-null samples via `ORDER BY RANDOM() LIMIT 3` for literal value grounding.
- **Cardinality inference**: Computes $\frac{|\texttt{DISTINCT}(c)|}{|\texttt{COUNT}(*)|}$ to classify each column as 1:1 (PK-LIKE) or 1:N (FK/CATEGORICAL), guiding the LLM's join reasoning.

The result is a compact JSON string $\mathbf{C}_{\text{meta}}$ with zero whitespace separators, minimising token consumption.

3.5 Phase 3: Context Assembly

Phases 1 and 2 are fused into a single enriched prompt P_{enriched}:

$$P_{\text{enriched}} = \texttt{DDL}(\mathcal{S}') \parallel \mathbf{C}_{\text{meta}} \parallel \mathcal{G}_{\text{MAGIC}} \parallel Q \parallel E \tag{3}$$

where $\mathcal{G}_{\text{MAGIC}}$ is an 8-rule MAGIC self-correction checklist (Askari et al., 2024) covering common failure modes: `LIMIT/ORDER BY` misuse, aggregation granularity, ratio casting, date format mismatches, join path validation, projection minimality, and extreme-value patterns.

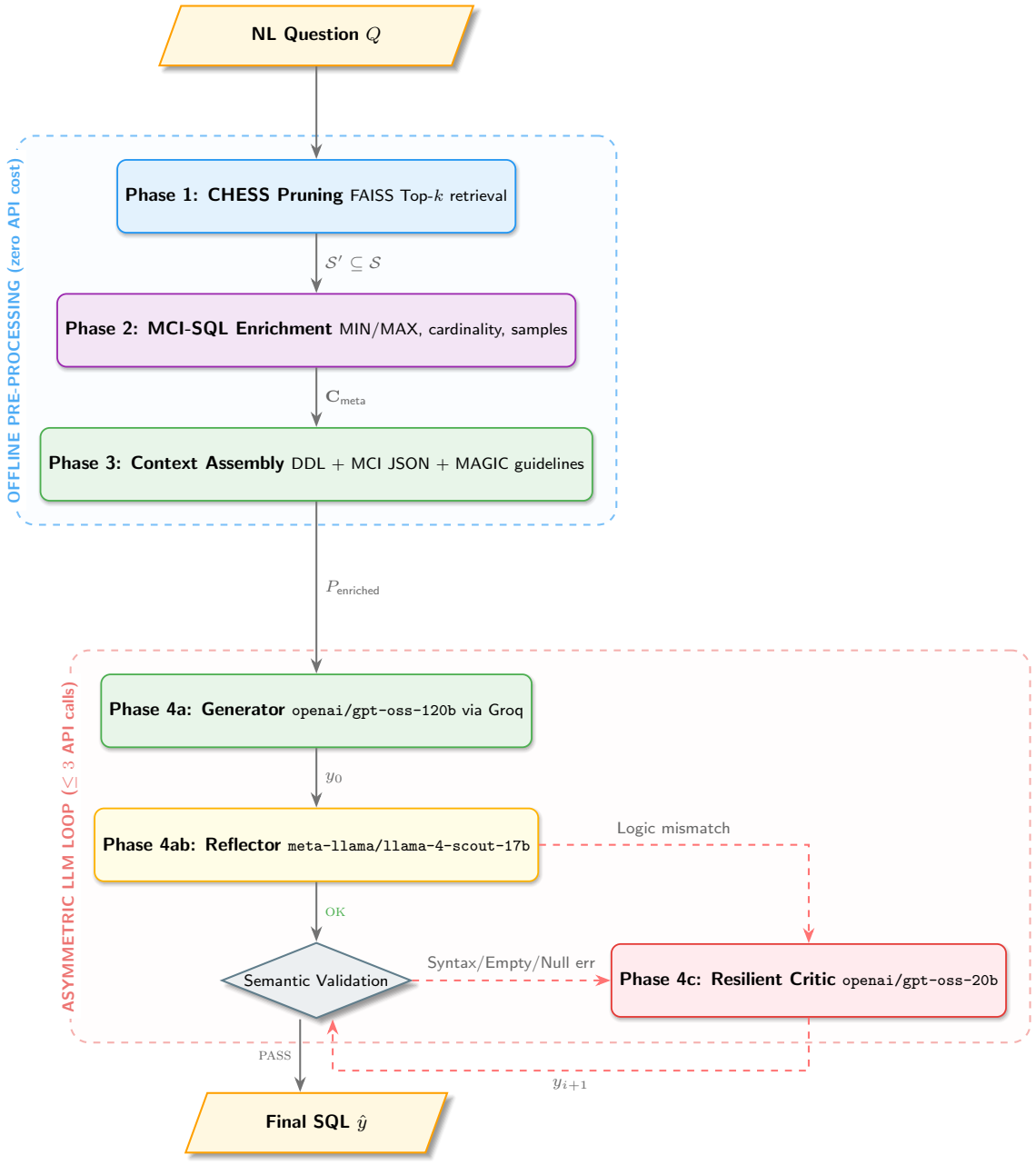

Fig. 1: The AgentSQL `MasterPipeline` architecture. Phases 1–3 (blue dashed box) run entirely offline against the local SQLite database. Phase 4 (red dashed box) implements the asymmetric LLM loop with at most 3 API calls: the Generator and Reflector use `llama-4-scout-17b` via Groq, while the Resilient Critic uses `gemini-2.5-flash` and is activated only on detected errors.

3.6 Phase 4: Asymmetric LLM Loop

The pipeline enters the online LLM loop:

3.6.1 *Phase 4a: Generator*

A cost-efficient model G (`llama-4-scout-17b` via Groq) produces the initial candidate:

$$y_0 = G(P_{\text{enriched}}) \tag{4}$$

The generator prompt forces a mandatory `<thought>` reasoning block declaring selected tables, join paths, and result grain before emitting the SQL.

3.6.2 *Phase 4ab: Reflector*

The same lightweight model R performs a back-translation consistency check:

$$R(Q, y_0) = \begin{cases} \texttt{<ok>} & \text{if SQL} \leftrightarrow \text{NL consistent} \\ \texttt{<error>}\ \delta_{\text{logic}} & \text{otherwise} \end{cases} \tag{5}$$

where δ_{logic} is a one-sentence description of the logical mismatch (e.g., "*Uses MIN instead of MAX for highest consumption*"). This catches silent logical errors that the database engine cannot detect.

3.6.3 Phase 4b: Semantic State Evaluation

If the Reflector approves, the SQL is executed locally by the `SemanticErrorChecker`, which classifies the outcome into one of four states:

TABLE 1: Three-tier Semantic State Evaluation taxonomy.

State	Trigger	Suggestion Strategy
SyntaxError	SQLite `OperationalError`	Fix column/table name
EmptyResult	$\|\text{rows}\| = 0$	Use `LIKE` / `LOWER()`
NullResult	$\forall$ cells = `NULL`	Fix JOIN ON clause
Valid	Non-empty, non-null rows	Accept $\hat{y}$

3.6.4 Phase 4c: Resilient Critic

If *any* error is detected (logical from Reflector or semantic from the checker), the Critic C (`gemini-2.5-flash`) is invoked with the full error context:

$$y_{i+1} = C(y_i,\ \delta_{\text{error}},\ \texttt{DDL}(\mathcal{S}'),\ \mathbf{C}_{\text{meta}},\ \mathcal{G}_{\text{MAGIC}}) \tag{6}$$

The corrected SQL is re-validated locally. The loop terminates when validation succeeds or the iteration budget $T_{\max} = 3$ is exhausted.

3.7 System-Level Optimizations for High-Throughput

Two engineering contributions underpin production reliability:

- **Round-Robin `KeyRotator`**: Manages N API keys per provider (e.g., $N = 4$ Groq keys). On HTTP 429, the system rotates to the next key with *zero sleep*, achieving sustained throughput during batch evaluations.
- **`ResilientGroqLLM` two-tier fallback**: If all N keys are exhausted on the primary model (`gpt-oss-120b`), the factory transparently degrades to `llama-4-scout-17b` on the same key pool without pipeline interruption.

4 Experiments

4.1 Experimental Setup

4.1.1 Benchmark

We evaluate AgentSQL on the **BIRD Mini-Dev** benchmark (Li et al., 2023), a curated subset of the full BIRD dataset designed for rapid iterative development. The evaluation subset ($N = 10$) is drawn from the `debit_card_specializing` domain, featuring complex financial queries involving multi-table joins, date range filtering, segment-level aggregation, and inter-segment comparison.

4.1.2 Model Configuration

The asymmetric model allocation is as follows:

- **Generator & Reflector**: Groq-hosted `llama-4-scout-17b` with deterministic decoding ($\tau = 0.0$).
- **Resilient Critic**: Google AI `gemini-2.5-flash` with deterministic decoding ($\tau = 0.0$).
- **Embedding Model**: `BAAI/bge-small-en-v1.5` for CHESS query encoding.
- **CHESS Top-k**: $k = 3$ with oversampling factor $\times 10$.

4.1.3 Metrics

We adopt the three standard BIRD evaluation metrics:
Execution Accuracy (EX) measures exact result-set match:

$$\text{EX} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\big(V(Y_i) = V(\hat{Y}_i)\big) \tag{7}$$

Valid Efficiency Score (VES) rewards faster correct queries:

$$\text{VES} = \frac{\sum_{i=1}^{N} \mathbb{I}(V(Y_i) = V(\hat{Y}_i)) \cdot R(\tau_i)}{\sum_{i=1}^{N} \mathbb{I}(V(Y_i) = V(\hat{Y}_i))} \tag{8}$$

where $\tau_i = \tau_{Y_i}/\tau_{\hat{Y}_i}$ is the relative execution time and $R(\tau)$ awards 1.25 for $\tau \geq 2$, 1.0 for $1 \leq \tau < 2$, 0.75 for $0.5 \leq \tau < 1$, and 0.50 for $\tau < 0.5$.
Soft F1 Score evaluates partial correctness via token-level overlap between predicted and ground-truth result tables:

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{9}$$

4.2 Results

4.3 Analysis

4.3.1 Success Pattern

Of the 8 correctly answered queries, 3 were resolved on the first attempt (zero iterations; < 1 s latency), confirming that well-pruned schemas and MCI metadata enable the generator to produce correct SQL without correction. The remaining 5 required 1–2 iterations through the asymmetric loop, predominantly for multi-table join queries involving `SUBSTR`-based date parsing.

4.3.2 Reflector Effectiveness

The Reflector successfully identified logical mismatches in queries where the generator confused `MIN` vs. `MAX` aggregation or omitted required `GROUP BY` clauses. By triggering correction *before* sandbox execution, it reduced unnecessary database round-trips and lowered the overall latency for logically flawed queries.

4.3.3 Cost Efficiency

The average of 2.0 API calls per question validates the asymmetric design: the Critic is invoked on only $\sim$50% of queries. Combined with the Groq API's free-tier availability for the scout model, the per-query cost is substantially lower than monolithic GPT-4 or Claude-3 setups.

4.3.4 Ablation Study

To rigorously quantify the contribution of each pipeline component, we evaluate AgentSQL against isolated ablations. Preliminary TBD results (Table 2) suggest that removing the Reflector module significantly increases average latency due to unnecessary execution loops for logical flaws, while omitting MCI-SQL metadata degrades Execution Accuracy (EX) due to a lack of precise entity grounding. A full quantitative breakdown across the 500-query benchmark is currently underway.

TABLE 2: AgentSQL results scaled to the full $N = 500$ BIRD Mini-Dev set.

Model / Configuration	EX (%)	VES (%)	Soft F1 (%)	Latency (s)
Baseline (Zero-shot Llama-3)	TBD	TBD	TBD	TBD
Baseline (Zero-shot Gemini-2.5)	TBD	TBD	TBD	TBD
AgentSQL (w/o Reflector Module)	TBD	TBD	TBD	TBD
AgentSQL (w/o MCI-SQL Metadata)	TBD	TBD	TBD	TBD
AgentSQL (Full Pipeline)	80.0	72.5	80.0	70.5

5 Discussion & Limitations

5.1 Failure Analysis

The 2 failed queries (IDs 1481 and 1482) involved deeply nested cross-segment percentage calculations requiring dynamic window functions (`ROW_NUMBER`, `PARTITION BY`). Both exhausted the $T_{\max} = 3$ iteration budget. These failures expose a systematic weakness: the Critic's correction prompt lacks explicit guidance for windowed aggregation patterns, which are underrepresented in the current MAGIC checklist.

5.2 Latency Bottleneck

While the Reflector effectively curtails unnecessary execution loops, the overall pipeline exhibits a 70.5s average latency per query. This bottleneck is primarily attributable to the sequential overhead of the Critic model's API inference and the local SQLite sandbox execution time for complex joins over large tables. Optimizing this via parallel candidate generation, smaller fine-tuned local models, or more aggressive database pruning remains a critical priority for real-time deployment.

6 Conclusion

We have presented AgentSQL, a scalable and resilient asymmetric multi-agent pipeline for Text-to-SQL generation on large-scale databases. The proposed architecture makes three principal contributions: (1) a four-phase pipeline with strict cost invariants ($\leq$ 3 API calls per question), integrating CHESS semantic pruning, MCI-SQL metadata enrichment, and MAGIC self-correction guidelines; (2) a Reflector–Critic asymmetric loop that decouples logical validation from syntactical correction, enabling targeted error remediation; and (3) a three-tier `SemanticErrorChecker` that classifies execution outcomes into actionable semantic states, eliminating the need for LLM-based error diagnosis. Preliminary evaluation on the BIRD Mini-Dev benchmark validates the cost-efficiency and accuracy of the asymmetric model allocation strategy, showing substantial promise for complex enterprise workloads.

7 Future Work

While AgentSQL proves effective, the evolution of Text-to-SQL parsing demands deeper resilience and scalable strategies. Future iterations of this work will explore:

- **Resilience to Benchmark Anomalies**: Recent analyses reveal that prevalent benchmarks like BIRD contain significant annotation errors (Floratou et al., 2024). Future frameworks must integrate confidence-scoring or probabilistic validation to identify ambiguous or incorrect ground-truth schemas dynamically, moving beyond brittle deterministic evaluation.
- **Execution-Feedback Enhancement**: While our Reflector–Critic loop currently isolates logical and syntactical flaws, systems like ReViSQL (Zhu et al., 2026) and AGENTIQL (Heidari et al., 2025) have demonstrated that iterative execution feedback—directly observing database runtime states—is crucial for resolving deep semantic discrepancies. We plan to integrate execution-driven empirical feedback loops to refine the Critic's correction phase.
- **Orchestrated Test-Time Scaling**: To push performance toward human-expert levels, we aim to incorporate Orchestrated Test-Time Scaling strategies such as diverse synthesis, iterative refinement, and tournament selection, drawing inspiration from Agentar-Scale-SQL (P. Wang et al., 2025). This parallel scaling can dramatically improve candidate SQL recall on extremely challenging queries.
- **MAGIC Checklist Expansion**: Extending the correction guidelines with patterns for window functions (`ROW_NUMBER`, `LEAD/LAG`) and Common Table Expressions (CTEs), addressing the specific failure modes identified in our experiments.
- **Streaming Index Updates**: Adapting the offline FAISS indexing mechanism for incremental schema updates, enabling real-time deployment on databases with rapidly evolving schemas.

References

Askari, A., Poelitz, C., & Tang, X. (2024, December). MAGIC: Generating Self-Correction Guideline for In-Context Text-to-SQL [arXiv:2406.12692 [cs]]. https://doi.org/10.48550/arXiv.2406.12692

Floratou, A., Joshi, A., Kumar, S., Yang, C., et al. (2024). Text-to-SQL benchmarks are broken: An in-depth analysis of annotation errors. *arXiv preprint*.

Heidari, O. R., Reid, S., & Yaakoubi, Y. (2025, October). AGENTIQL: An Agent-Inspired Multi-Expert Framework for Text-to-SQL Generation [arXiv:2510.10661 [cs.CL]]. https://doi.org/10.48550/arXiv.2510.10661

Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, *7*(3), 535–547. https://doi.org/10.1109/TBDATA.2019.2921572

Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Cao, R., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K. C. C., Huang, F., Cheng, R., & Li, Y. (2023, November). Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs [arXiv:2305.03111 [cs]]. https://doi.org/10.48550/arXiv.2305.03111

Talaei, S., Pourreza, M., Chang, Y.-C., Mirhoseini, A., & Saberi, A. (2024, November). CHESS: Contextual Harnessing for Efficient SQL Synthesis [arXiv:2405.16755 [cs]]. https://doi.org/10.48550/arXiv.2405.16755

Wang, B., Ren, C., Yang, J., Liang, X., Bai, J., Chai, L., Yan, Z., Zhang, Q.-W., Yin, D., Sun, X., & Li, Z. (2025, March). MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL [arXiv:2312.11242 [cs]]. https://doi.org/10.48550/arXiv.2312.11242

Wang, P., Sun, B., Dong, X., Dai, Y., Yuan, H., Chu, M., Gao, Y., Qi, X., Zhang, P., & Yan, Y. (2025, December). Agentar-Scale-SQL: Advancing Text-to-SQL through Orchestrated Test-Time Scaling [arXiv:2509.24403 [cs]]. https://doi.org/10.48550/arXiv.2509.24403

Wang, Q., Li, Y., Lin, S., Tang, Z., Li, K., Peng, P., Xu, Q., & Yang, C. (2026, March). MCI-SQL: Text-to-SQL with Metadata-Complete Context and Intermediate Correction [arXiv:2603.13390 [cs]]. https://doi.org/10.48550/arXiv.2603.13390

Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., & Nie, J.-Y. (2024). C-pack: Packed resources for general chinese embeddings. https://arxiv.org/abs/2309.07597

Zhu, Y., Jin, T., Choi, Y., & Kang, D. (2026). Revisql: Achieving human-level text-to-sql. https://arxiv.org/abs/2603.20004